

RETAIL AI REORDER PREDICTION SYSTEM

1. Project Overview

This project is an end-to-end machine learning engineering system for retail reorder prediction using an Instacart-style dataset. It predicts whether a customer is likely to reorder a product based on order history, product behavior, and cart-level signals.

The project is designed to demonstrate more than model training. It shows how a production-style ML system is structured: data ingestion, feature engineering, training, experiment tracking, model serving, monitoring, CI/CD, and a GenAI assistant layer.

2. Business Problem

Retail and grocery platforms need to understand which products a customer is likely to buy again. This helps with:

- personalized recommendations
- buy-again sections
- targeted promotions
- inventory planning
- customer retention

The core prediction question is:

Will this customer reorder this product?

The model output is:

- 1: likely reordered
- 0: not likely reordered

3. End-to-End Workflow

The system follows this workflow:

Raw Instacart data

->

Ingestion pipeline

->

Processed data tables

->

Feature engineering

->

Training dataset

->

ML model training

->

MLflow tracking

->

Model serving API

->

Prediction + GenAI explanation

->

Monitoring and retraining trigger

4. Project Structure

config/	Declarative YAML configs
data/raw/	Full local Instacart data
data/sample/raw/	Small demo dataset
data/processed/	Processed tables
data/features/	Feature tables
data/predictions/	Metrics, predictions, saved model
src/ingestion/	Data ingestion logic
src/processing/	Data joining and transformation
src/features/	Feature engineering
src/models/	Training, evaluation, prediction
src/pipelines/	Declarative pipeline runner
src/serving/	FastAPI serving layer
src/rag/	RAG/GenAI assistant foundation
src/monitoring/	Drift and retraining checks
tests/	Unit tests
.github/workflows/	GitHub Actions CI

5. Tools And Frameworks

Python

Python is used as the main programming language because it has strong support for data engineering, machine learning, APIs, and GenAI integrations.

PySpark

PySpark is used for the scalable data-processing path. It is suitable for large datasets because it can distribute reads, joins, aggregations, and writes across a Spark cluster.

In this project, PySpark is used for:

- reading large CSV files

- converting raw data to Parquet
- joining orders, products, and order-product data
- aggregating product and user features

PySpark requires Java because the Spark execution engine runs on the JVM. Python controls Spark, but Spark itself executes through the JVM.

Pandas

Pandas is used as a lightweight demo engine for small local data. This allows the project to run without Java or Spark setup during demos.

The demo mode is activated with:

```
$env:RETAIL_ENGINE="pandas"
```

This is useful for fast local demonstrations and CI-friendly testing.

Parquet

Parquet is used to store processed and feature data. It is columnar, compressed, and commonly used in data lakes and ML pipelines.

In production, this could be upgraded to Delta Lake for ACID transactions, schema enforcement, and time travel.

Scikit-Learn

Scikit-learn is used for the baseline ML model. The current model is a Random Forest classifier.

Random Forest is used because it works well as a strong baseline for tabular data and can capture non-linear relationships between customer behavior and reorder probability.

MLflow

MLflow is used for model lifecycle management.

In this project, MLflow handles:

- experiment tracking
- parameter logging
- metric logging
- model artifact logging where supported
- model lifecycle structure

The project currently trains one model type: Random Forest.

FastAPI

FastAPI is used to expose the trained model through an API.

Endpoints include:

- /health: service health check
- /predict: reorder prediction
- /assistant: GenAI explanation
- /demo: simple browser-based demo UI
- /docs: Swagger API documentation

FastAPI is used because it is fast, modern, easy to document, and production-friendly.

Joblib

Joblib is used to save and load the locally trained model as:

```
data/predictions/retail_model.joblib
```

This allows the FastAPI app to serve the sample-trained model locally without requiring the MLflow model registry.

Grok / xAI API

The GenAI assistant can call Grok using the xAI API when an API key is provided.

The assistant is used for:

- explaining predictions in business language
- summarizing why a product may be reordered
- making the output easier for business users to understand

Environment variables:

```
$env:XAI_API_KEY="your_key"  
$env:XAI_MODEL="grok-4"
```

If no key is provided, the assistant uses a local fallback explanation.

GitHub Actions

GitHub Actions is used for CI/CD validation.

It runs tests on push and pull request to ensure the codebase remains healthy.

Render

Render is used to host the lightweight FastAPI demo.

The Render deployment uses demo mode because the full Instacart dataset and trained local model artifacts are not pushed to GitHub.

6. Feature Engineering

The feature engineering layer creates features such as:

- order_number
- order_dow
- order_hour_of_day
- days_since_prior_order
- add_to_cart_order
- user_order_count
- product_order_count
- product_reorder_rate
- aisle_id
- department_id

These features represent customer behavior, product popularity, order timing, and product category.

Example interpretation:

product_reorder_rate = 0.6

means the product is reordered 60 percent of the time in historical data.

7. Model Training

The model training step reads the feature table and trains a Random Forest classifier.

Training includes:

- train-test split
- numeric feature imputation
- categorical feature encoding
- model training
- accuracy calculation
- ROC-AUC calculation
- MLflow metric logging
- validation prediction export

Example output:

{

```
"accuracy": 0.8,  
"roc_auc": 0.75  
}
```

8. Model Serving

The trained model is served with FastAPI.

Sample request:

```
{  
  "records": [  
    {  
      "order_number": 10,  
      "order_dow": 1,  
      "order_hour_of_day": 5,  
      "days_since_prior_order": 7,  
      "add_to_cart_order": 1,  
      "user_order_count": 5,  
      "product_order_count": 100,  
      "product_reorder_rate": 0.6,  
      "aisle_id": 24,  
      "department_id": 4  
    }  
  ]  
}
```

Sample response:

```
{  
  "predictions": [  
    {  
      "prediction": 1,  
      "label": "likely_reordered",  
      "confidence_hint": "model_score",  
      "reason": "Generated by the ML model."  
    }  
  ],  
  "mode": "local_joblib"  
}
```

9. GenAI Assistant

The GenAI assistant is not the prediction model. It is an explanation layer around the ML system.

The ML model predicts:

likely reordered or not likely reordered

The GenAI assistant explains:

why the prediction matters in business language

This is relevant in production because business users often need explanations, not just raw model outputs.

10. Monitoring And Self-Improving Loop

The monitoring module checks whether the model metric falls below a threshold.

Example:

```
if accuracy < 0.75:  
    retrain pipeline
```

This is a simple version of a self-healing ML pipeline. In production, this can be expanded with feature drift, data quality checks, model performance monitoring, and scheduled retraining.

11. Local Demo Commands

Train with the small demo dataset:

```
$env:RETAIL_ENGINE="pandas"  
$env:RETAIL_RAW_DIR="data/sample/raw"  
$env:RETAIL_TRAINING_CONFIG="config/training_demo.yaml"  
python main.py
```

Start FastAPI with the locally trained model:

```
$env:USE_MLFLOW_MODEL="false"  
$env:DEMO_MODE="false"  
$env:LOCAL_MODEL_PATH="data/predictions/retail_model.joblib"  
python -m uvicorn src.serving.api:app --host 127.0.0.1 --port 8002
```

Open:

<http://127.0.0.1:8002/demo>

12. Production Mapping

This project maps to production systems as follows:

Project Component	Production Equivalent
-------------------	-----------------------

--- ---

data/raw	Data lake storage such as S3, ADLS, or GCS
PySpark pipeline	Distributed ETL or Databricks job
Parquet feature table	Offline feature store
MLflow tracking	Experiment tracking and model registry
FastAPI	Real-time model serving API
GenAI assistant	LLM-based business explanation layer
Drift check	Monitoring and retraining trigger
GitHub Actions	CI/CD validation
Render	Lightweight API hosting

13. Why This Project Is Relevant

This project demonstrates the responsibilities of a machine learning engineer:

- building data and feature pipelines
- training and evaluating models
- tracking experiments
- serving predictions through APIs
- adding GenAI explanations
- validating code with CI/CD
- designing for both local demo and scalable production paths

It is a simplified but realistic version of a production retail AI system.